



Universidad autónoma del estado de Mexico
Facultad de ingeniería



“Ingeniería en Inteligencia Artificial”

“Programacion II”

“Proyecto”

Equipo:

Los Skibidi Toilet

Integrantes:

Axel Yael Cruz Cruz

Galia Gomez Lopez

Valeria Muñoz Guiza

Erick Emmanuel Romero Mejía

Toluca mexico a 30 de mayo de 2026

Introducción:

El procesamiento y la validación de actas electorales representan uno de los mayores desafíos logísticos y tecnológicos en los procesos democráticos contemporáneos. Históricamente, la digitalización y el conteo de estos documentos se han realizado de manera manual o mediante sistemas rígidos, lo que expone el proceso a errores humanos, retrasos en la entrega de resultados y dificultades operativas debido a la calidad variable de los insumos físicos (como imágenes borrosas, problemas de iluminación o documentos arrugados y desalineados).

Para resolver esta problemática, el presente proyecto, desarrollado por estudiantes de la Universidad Autónoma del Estado de México, propone el diseño e implementación de un sistema automatizado para la digitalización y extracción de votos en actas electorales. Utilizando la tecnología de Azure AI Document Intelligence, el software emplea modelos preentrenados de Inteligencia Artificial capaces de reconocer y extraer texto de manera precisa, sin importar las imperfecciones físicas del archivo.

La arquitectura del sistema destaca por su enfoque modular y el uso de tecnologías eficientes como FastAPI para el backend, garantizando respuestas en milisegundos. Además, integra librerías avanzadas como *difflib* para corregir errores ortográficos mediante similitud de palabras, y *statistics* para recalcular y alinear matemáticamente las filas de votación en caso de lecturas desfasadas. Finalmente, el sistema ofrece una interfaz de usuario minimalista desarrollada con Chart.js, transformando los datos crudos en inteligencia visual inmediata (gráficas de barras y de pastel) para optimizar la toma de decisiones. Con esto, se presenta una herramienta tecnológica transparente, neutral y altamente adaptable a los cambios regulatorios del entorno electoral.

Objetivos Específicos:

- **Implementar Inteligencia Artificial para la extracción de datos:** Integrar el servicio de *Azure AI Document Intelligence* para identificar y extraer la información de las actas electorales, asegurando una alta tolerancia a fallas físicas como imágenes mal iluminadas, borrosas o desalineadas.
- **Desarrollar un backend de alta velocidad y seguro:** Construir la API del sistema utilizando *FastAPI* y *CORSMiddleware* para lograr un procesamiento de datos en milisegundos y una comunicación segura entre los diferentes módulos del software.
- **Garantizar la pulcritud y consistencia de los datos:** Utilizar librerías de procesamiento lógico y matemático (*difflib* y *statistics*) para limpiar el texto, corregir errores ortográficos por aproximación y alinear estadísticamente las filas de votación correspondientes a cada partido o alianza.
- **Diseñar un módulo de reconciliación y auditoría:** Desarrollar algoritmos (*reconciliacion.py* y *utils.py*) que sumen de forma automatizada los votos extraídos de cada partido, verifiquen de manera lógica que coincidan con el total general reportado y auditen el proceso mediante una doble verificación (número y letra) para evitar incongruencias.
- **Visualizar los resultados de manera inmediata:** Crear una interfaz de usuario ágil y minimalista que emplee *Chart.js* para renderizar de forma instantánea gráficas de barras y de pastel, facilitando la interpretación visual de la información electoral recopilada.
- **Asegurar la escalabilidad del sistema:** Diseñar el software bajo una arquitectura modular y parametrizada (*config.py*, *coaliciones.py*, *diccionarios.py*) que permita adaptar el programa fácilmente a futuras alianzas, nuevos partidos o modificaciones en las reglas del conteo electoral.

Explicación del código:

config.py Configura la conexión con el servicio de Azure Form Recognizer (IA para leer documentos). Define el endpoint, la clave de autenticación y parámetros globales como tolerancia de filas, umbral de coincidencia difusa y modo debug.

Diccionarios.py: Define tres diccionarios de datos estáticos: conversión de números escritos en palabras a dígitos (mapa_numeros), normalización de nombres de partidos a su forma oficial (normalizar), y el orden canónico en que deben aparecer los partidos en el acta (orden_partidos).

Utils.py: Contiene funciones de utilidad reutilizables: palabra_a_numero convierte texto como "veinte" al número 20; cell_center obtiene las coordenadas del centro de una celda detectada; fuzzy_normalize busca el nombre de partido más parecido usando coincidencia aproximada; y extraer_digitos extrae únicamente los dígitos de un texto.

Coaliciones.py: Interpreta el texto crudo de una celda para identificar si contiene un partido individual o una coalición. Tokeniza el texto, mapea cada token a su partido oficial y, si hay varios partidos, los une en un nombre canónico con guión (ej. "Morena-PT").

Reconciliacion.py: Intenta corregir inconsistencias en el conteo de votos. Compara la suma del desglose con el total oficial del acta y, si hay diferencia, prueba reasignar votos de filas sin partido identificado hacia sus filas vecinas, o ajusta valores sospechosos si la diferencia es pequeña (≤ 5 votos).

Acta.py: Es el módulo central. Define la clase Acta (con atributos de casilla, total y desglose) y la función procesar_acta, que envía la imagen a Azure, extrae las celdas de la tabla detectada, identifica partidos y votos por fila, calcula un offset para corregir desalineaciones, y devuelve un objeto Acta con el desglose reconciliado.

Server.py: Expone la funcionalidad como una API REST usando FastAPI. Recibe una imagen de acta por HTTP POST en /procesar_acta/, la guarda temporalmente, llama a procesar_acta y devuelve en JSON el id de casilla, el total de votos y el desglose por partido (filtrando filas basura).

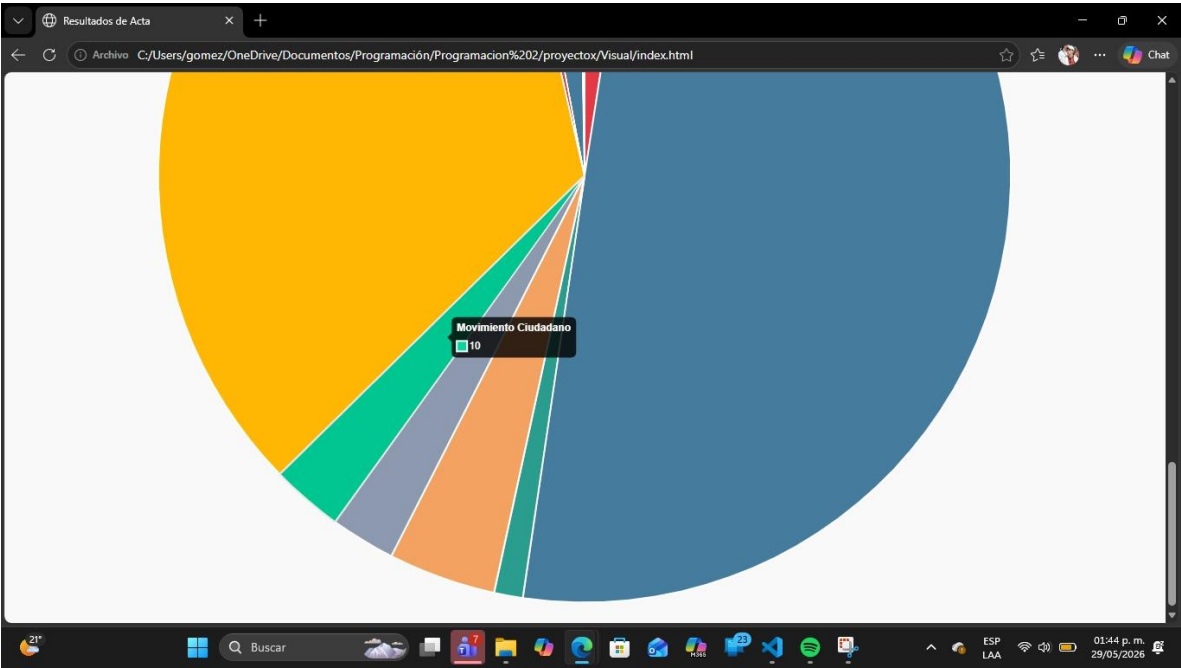
index.html: Es la interfaz web del proyecto. Permite subir una imagen del acta, enviarla al servidor, y mostrar los resultados en una tabla con porcentajes y dos gráficas (barras y pastel) generadas con Chart.js.

Evidencia:

The screenshot displays a Visual Studio Code editor window with a project named 'proyecto2'. The Explorer sidebar on the left shows the project structure, including files like 'main.py', 'server.py', 'index.html', 'acta.py', 'reconciliacion.py', 'utils.py', and 'coaliciones.py'. The main editor area shows the 'server.py' file, which contains an asynchronous function 'procesar' that takes an 'UploadFile' object and a 'partido' string as input. The function returns a dictionary with 'id_casilla', 'total_votos', and 'desglose_partidos' fields. Below the code editor, the TERMINAL panel shows the command prompt output, indicating that the application is running on http://127.0.0.1:8000 and that the server process has started successfully.

Below the editor, a web browser window is open, displaying the 'Resultados de Acta' page. The page has a title 'Resultados de Acta' and a subtitle 'Elegir archivo No se eligió ningún archivo Procesar Acta'. Below this, there is a table with three columns: 'Partido', 'Votos', and 'Porcentaje'. The table is currently empty, and the browser's address bar shows the URL 'C:/Users/gomez/OneDrive/Documents/Programación/Programación%202/proyecto2/Visual/index.html'.

Partido	Votos	Porcentaje
---------	-------	------------



Resultados de Acta

Elegir archivo: acta1.jpg Procesar Acta

Partido	Votos	Porcentaje
PAN	9	2.45%
PRI	183	49.86%
PRD	4	1.09%
PVEM	15	4.09%
PT	9	2.45%
Movimiento Ciudadano	10	2.72%
Morena	124	33.79%
Nueva Alianza	2	0.54%
PVEM-PT-Morena	10	2.72%
PVEM-PT	1	0.27%
PVEM-Morena	0	0.00%
PT-Morena	0	0.00%
Candidatos/as no registrados/as	0	0.00%
Votos nulos	0	0.00%

